Feature Detection & Matching

Week 10 — Finding and comparing distinctive points

Practical Image Processing

Lecture 10 ▪ 60–70 min

Recap & today

- Wk5–9: edges, regions, shapes — describing an image's content.
- Today: **features** — distinctive points (corners) with **descriptors** you can **match** across images, even under rotation and scale.

DocScan milestone

Matching features between a photographed page and a template lets us align/rectify it — the setup for Week 11's perspective correction.

Roadmap

Corners: what makes a good feature

Descriptors

Matching

In-class exercise

Deep dive

Wrap-up

Corners: what makes a good feature

What is a good feature point?

- **Distinctive**: it looks different from its surroundings (so it can be found again).
- **Repeatable**: the same physical point is detected across images (different view/light).
- **Local**: described by a small patch, robust to partial occlusion.

Key idea

Flat regions and straight edges are ambiguous; **corners** are the classic good feature — they pin down a location in 2D.

Flat / edge / corner

- **Flat** patch: moving it any direction looks the same → no localization.
- **Edge**: moving *along* the edge looks the same → 1D ambiguity.
- **Corner**: moving *any* direction changes the patch → pinned in 2D.

Math

The **Harris** detector measures intensity change in all directions via the structure matrix's eigenvalues.

Harris corners — on a real car



grayscale



Harris corners (red)

Corners land on distinctive junctions (lights, edges of parts) — repeatable points to match on

The structure matrix, intuitively

- Look at how image gradients (I_x, I_y) vary inside a small window.
- Build the 2×2 matrix $M = \sum \begin{pmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{pmatrix}$.
- Its two eigenvalues λ_1, λ_2 say how much the patch changes in the two principal directions.

Math

Both small $\rightarrow$ flat. One large $\rightarrow$ edge. Both large $\rightarrow$ corner. Harris score $R = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2$ avoids computing eigenvalues directly.

Corner detectors

Detector	Idea	Note
Harris	eigenvalues of structure matrix	classic, accurate
Shi-Tomasi	min eigenvalue (goodFeaturesToTrack)	great for tracking
FAST	pixel-ring intensity test	very fast, used in ORB

Key idea

Detecting *where* the feature is (the keypoint) is step one. Describing *what it looks like* (the descriptor) is step two.

Descriptors

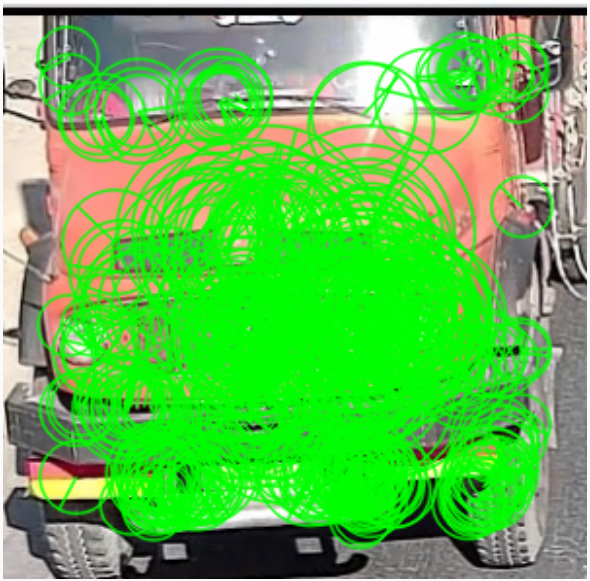
From keypoint to descriptor

- A **descriptor** is a vector summarizing the patch around a keypoint.
- Good descriptors are **invariant**: same value under rotation, scale, and lighting change.
- Match two images by finding keypoints with *similar descriptors*.

Math

SIFT: a 128-D vector of gradient-orientation histograms; robust to rotation/scale/illumination.

ORB keypoints — on the real car



SIFT vs ORB (and friends)

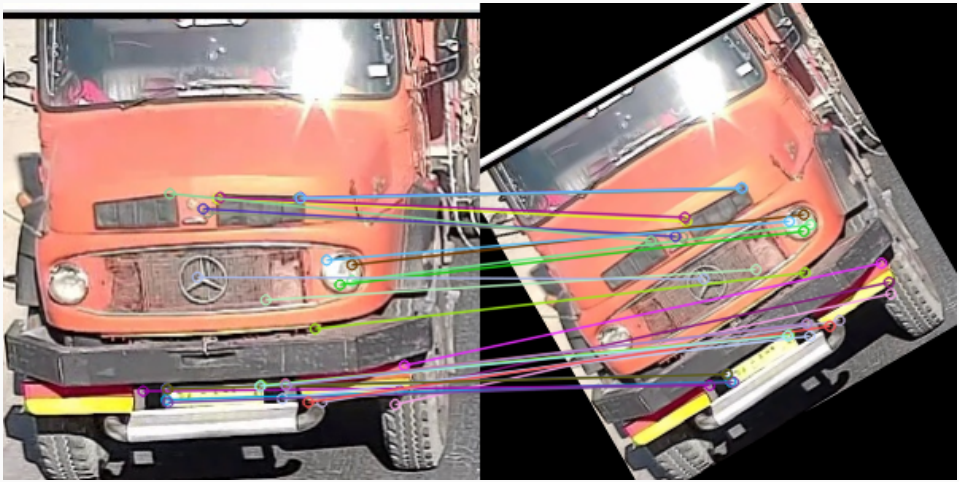
Descriptor	Type	Note
SIFT	128-D float	very robust; the gold standard
SURF	float	faster SIFT-like (patented)
ORB	binary (256-bit)	free, fast, great for real-time
BRIEF	binary	fast but not rotation-invariant alone

✔ Good practice

ORB is the practical default: free, fast, good enough. SIFT when you need maximum robustness.

Matching

Matching descriptors across images



Same car, rotated & scaled. ORB features still match — lines connect the same physical points. This is **invariance** working on real data.

How matching works

- For each descriptor in image A, find the **nearest** descriptor in image B (by distance).
- Float descriptors (SIFT): Euclidean distance. Binary (ORB): **Hamming** distance.
- **Brute-force** compares all pairs; **FLANN** approximates it fast for large sets.

Math

Hamming distance = number of differing bits — a single fast XOR+popcount for binary descriptors.

Rejecting bad matches

- Many nearest-neighbour matches are wrong. Filter them:
- **Ratio test** (Lowe): keep a match only if the best is much closer than the 2nd-best.
- **Cross-check**: keep only mutual nearest neighbours. **RANSAC** (Week 11): reject geometric outliers.

✕ Pitfall

Raw matches are noisy. Without the ratio test / RANSAC, a few wrong matches wreck the alignment.

In-class exercise

In-class exercise (10 min)

Reason about invariance

1. Why is a corner a better feature than a point in the middle of a flat wall?
2. You match a template plate to a photo taken at an angle and half the size. Which descriptor property makes this possible?
3. After matching, some lines cross wildly (obviously wrong). How do you remove them?

Debrief

- (1) A corner changes in *all* directions → uniquely localizable; a flat point is ambiguous.
- (2) **Scale and rotation invariance** — the descriptor is the same despite size/angle change.
- (3) Ratio test + cross-check remove most; **RANSAC** (Week 11) removes geometric outliers by fitting a transform.

Key idea

Good features + good matches + outlier rejection = reliable correspondence, the basis of alignment.

Deep dive

Scale invariance: the pyramid

- To find features at any size, build an **image pyramid** (progressively smaller copies).
- Detect features at each scale; the scale where a feature is strongest is its "characteristic scale."
- That's how SIFT/ORB match a small template to a large photo.

How many matches do you need?

- To estimate a **homography** (Week 11) you need ≥ 4 good correspondences.
- In practice, gather dozens of matches, then let **RANSAC** pick a consistent set of 4+.
- More good matches $\rightarrow$ a more stable, accurate transform.

Key idea

Quantity gives robustness: many matches let outlier rejection succeed even when a third are wrong.

Applications of features

- **Image stitching** (panoramas): match features, find the transform, blend.
- **Object recognition**: match a template's features to a scene.
- **Tracking, SLAM, AR**: follow features across video to estimate motion.
- **DocScan**: match the page/template to align and rectify (Week 11).

Classical features vs deep features

- Classical (SIFT/ORB): hand-designed, interpretable, no training — still excellent for geometry.
- Deep (learned descriptors, SuperPoint): better under extreme appearance change, need training.
- For alignment/geometry, classical features remain a strong, practical choice.

Key idea

Features are how a computer says "this point here is the same as that point there" — the foundation of geometric vision.

Features in code

```python
orb = cv2.ORB_create(500)
kp1, des1 = orb.detectAndCompute(img1, None)
kp2, des2 = orb.detectAndCompute(img2, None)
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = sorted(bf.match(des1, des2), key=lambda m: m.distance)
# next week: cv2.findHomography(pts1, pts2, cv2.RANSAC) to align
```

Detect $\rightarrow$ describe $\rightarrow$ match $\rightarrow$ (Week 11) estimate the transform.

Wrap-up

What you should now be able to do

- Explain what makes a good feature (distinctive, repeatable, local) and why corners.
- Detect corners (Harris/FAST) and compute descriptors (SIFT/ORB).
- Match descriptors; reject bad matches with the ratio test/cross-check.
- Understand scale/rotation invariance and the role in alignment.

This week's deliverable

Due before Week 11 (see `project/`)

1. Detect Harris corners and ORB keypoints on real images; visualize.
2. Match ORB features between an image and a rotated/scaled copy; count good matches.
3. Apply the ratio test / cross-check; show how it removes wrong matches.
4. Note: how will feature matching help align a photographed page in DocScan?

Features are “the same point, seen again.”

Detect distinctive points, describe them invariantly, match across images.